SD Karttan Boot (BeagleBone Black)

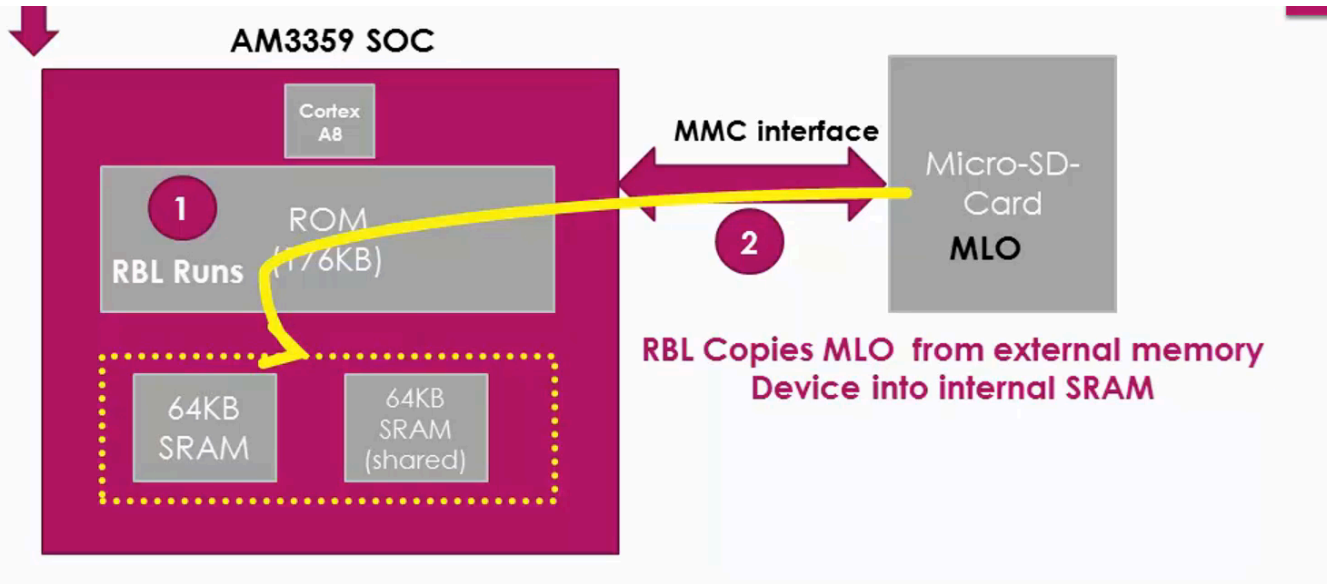
Bu dokümanda bir microSD kartı hazırlayıp BeagleBone Black'i **SD karttan Linux ile açacağız**.

Büyük Resim

Kart açılırken programlar sırayla birbirini çalıştırır. Buna **boot zinciri** denir:

```
Güç → RBL → MLO → U-Boot → Linux Kernel → Root Filesystem → login ekranı
```

Aşama	Ne yapar?	Nerede durur?
RBL (ROM Boot Loader)	Kart açılınca ilk çalışan kod. MLO'yu bulup çalıştırır.	İşlemcinin içinde, değiştirilemez
MLO	RAM'i (DDR) hazırlar, U-Boot'u yükler.	SD kart → BOOT bölümü
U-Boot	Kernel'i ve device tree'yi RAM'e yükleyip Linux'u başlatır.	SD kart → BOOT bölümü
Linux Kernel	İşletim sisteminin çekirdeği.	SD kart → ROOTFS bölümü (`/boot`)
Root Filesystem	Linux'un tüm dosyaları (`/bin` , `/etc` , ...).	SD kart → ROOTFS bölümü



SD kartın son hali şöyle olacak:

```
SD Kart
├── BOOT   (FAT16, ~1 GB)  →  MLO, u-boot.img, uEnv.txt
└── ROOTFS (ext3, kalan)   →  Linux dosyaları (kernel ve .dtb dahil)
```

> **Not**
> Boot zincirinin ayrıntıları: linux_booting_process.md

Yol haritası

Kursta dosyalar karta **tek tek** eklenir ve her seferinde kart açılıp test edilir. Böylece zincirin her halkasının çalıştığı görülür:

Adım	Ne yapılıyor?	Sonuç
1 – 3	Dosyaları indir, SD kartı bölümle	SD kart hazır
5	MLO'yu kopyala, test et	RBL ✓ → MLO ✓ → U-Boot ✗
6	U-Boot'u kopyala, test et	… → U-Boot ✓
7	Rootfs ve uEnv.txt'yi kopyala, test et	Linux açılır ✓
8	U-Boot'u durdur, uImage header'ını elle oku	Header çözüldü ✓

Her testte kart aynı şekilde açılır. Bu işlem Bölüm 4'te bir kez anlatılıyor.

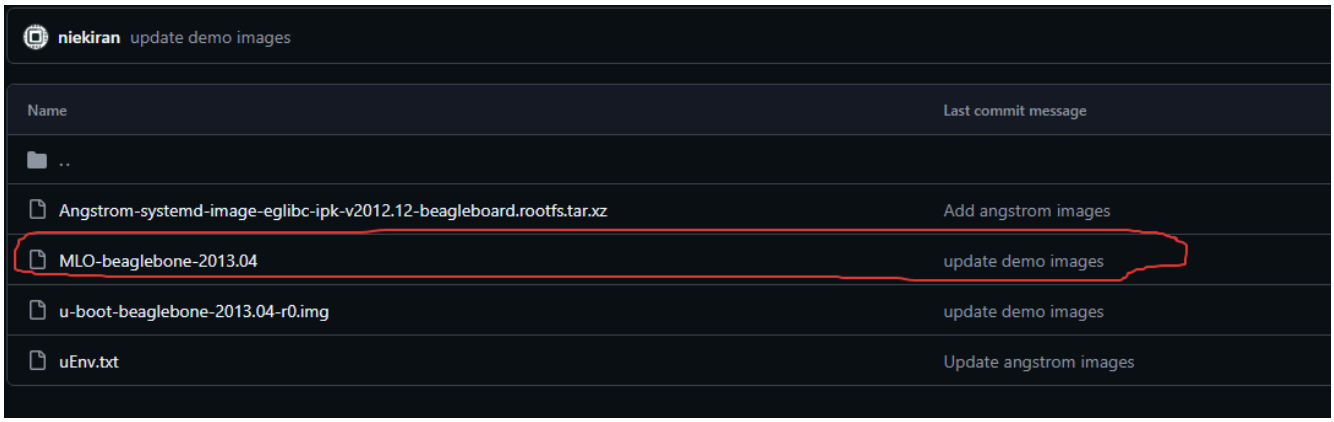
---

1. Hazırlık

Gerekenler:

- microSD kart (en az 4 GB) ve kart okuyucu
- USB-TTL seri dönüştürücü (3.3V). Kartın açılış mesajlarını görmek için gerekli.
- Mini USB kablo (karta güç vermek için)

Dosyaları indir. Kursun GitHub reposundaki **4 dosyanın hepsini** indir:
https://github.com/niekiran/EmbeddedLinuxBBB/tree/master/pre-built-images/Angstrom_Demo



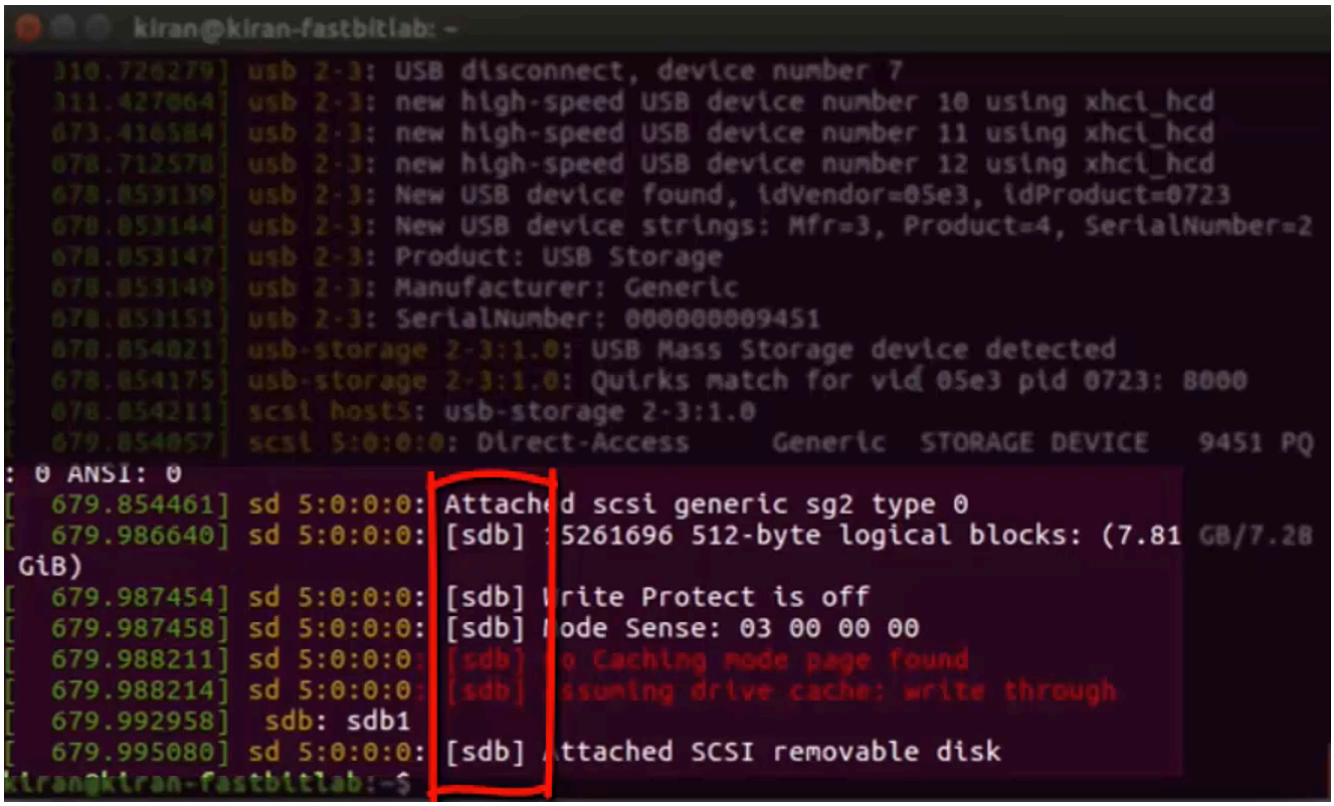
İndirilen dosya	Ne?	Nereye kopyalanacak?
`MLO-beaglebone-2013.04`	MLO	BOOT → adı `MLO` olacak
`u-boot-beaglebone-2013.04-r0.img`	U-Boot	BOOT → adı `u-boot.img` olacak
`uEnv.txt`	U-Boot ayar dosyası	BOOT → adı aynı kalacak
`Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz`	Root filesystem (içinde kernel `/boot/uImage` ve device tree `/boot/am335x-boneblack.dtb` da var)	ROOTFS → arşivden çıkarılacak

---

2. SD Kartı Bilgisayarda Bulma

Kartı takıp terminalde çalıştır:

```
dmesg
```



- `sdb` → SD kartın adı (`/dev/sdb`). Yanında boyutu yazar (7.81 GB). Doğru cihaz olduğunu boyuttan anlarsın.
- `sdb1` → kartta zaten var olan bölüm (partition).

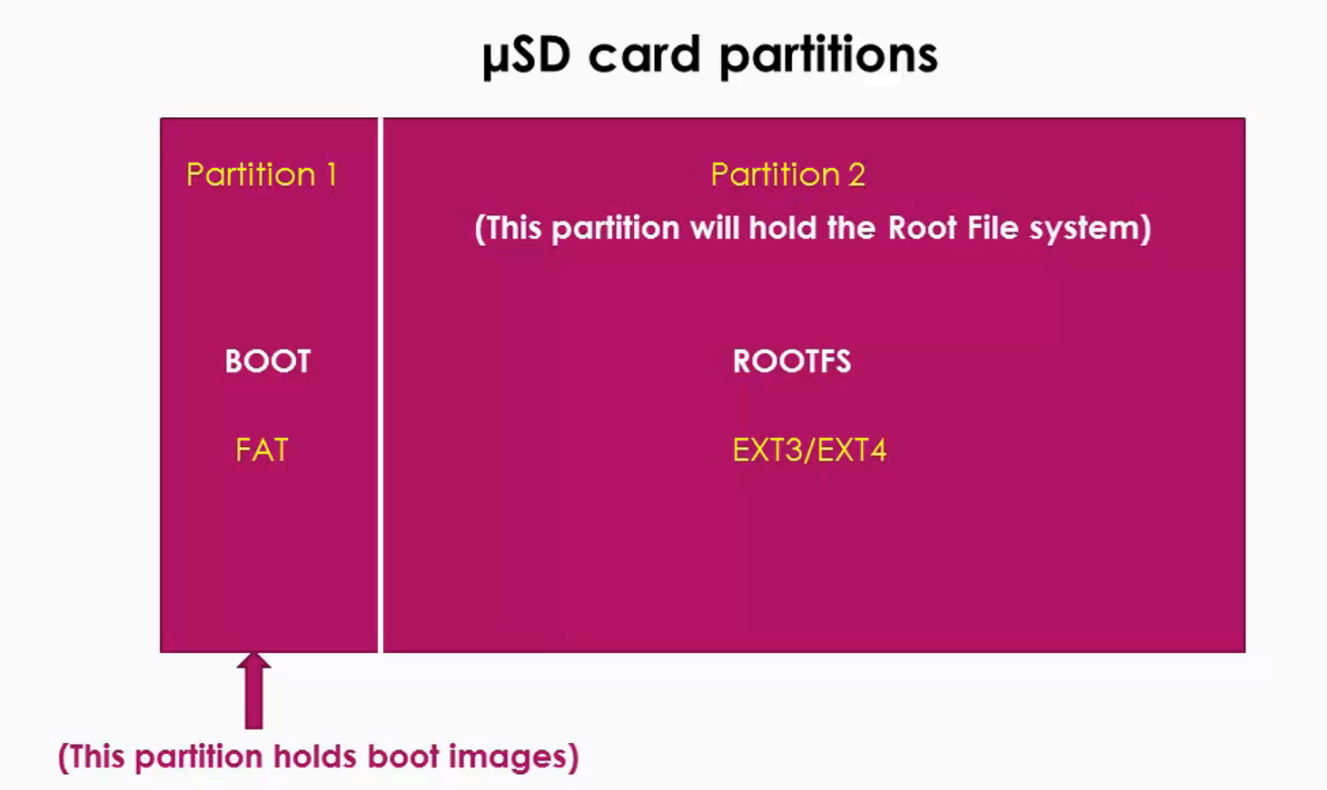
> **İpucu**
>
> `lsblk` komutu tüm diskleri boyutlarıyla birlikte daha okunaklı gösterir.

> **Not**
>
> GParted'da aynı kart **7.28 GiB** görünür. Kart eksik değil, birimler farklı: 1 GB = 1.000.000.000 byte, 1 GiB = 1.073.741.824 byte.

---

3. SD Kartı Bölümlere Ayırma (GParted)

Kart 2 bölüme ayrılacak. **Karttaki tüm veriler silinir.**

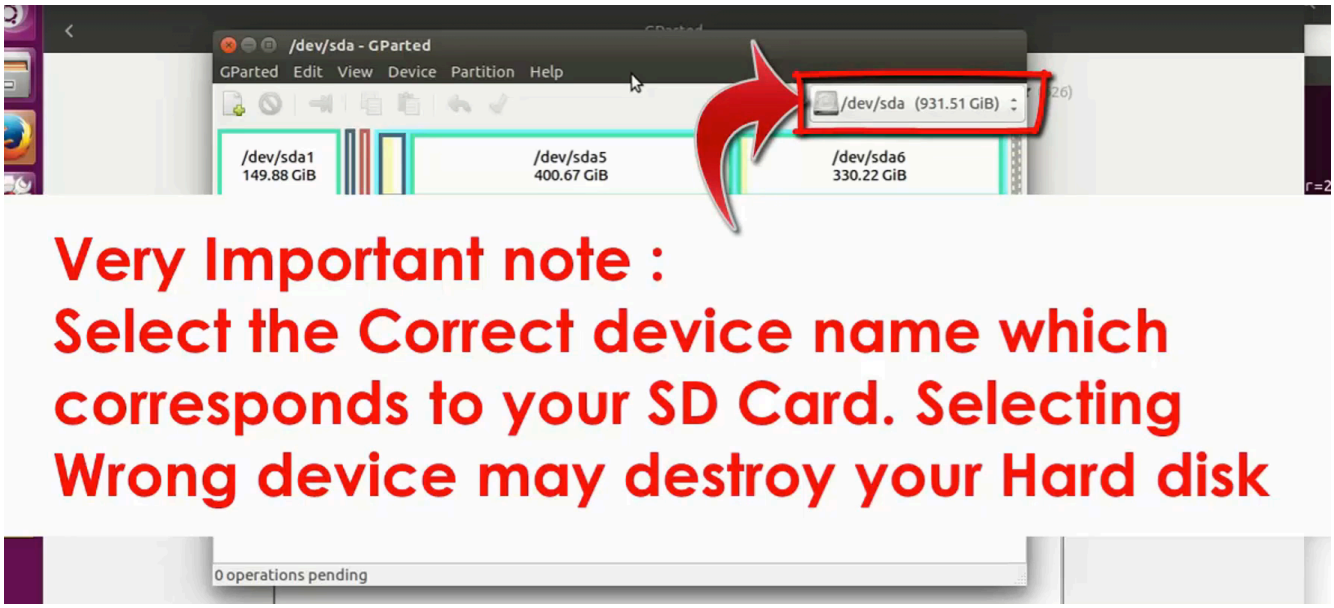


Bölüm	Label	Dosya sistemi	Boyut	Neden?
1	`BOOT`	`fat16`	~1 GB	RBL, MLO'yu sadece FAT bölümde arar
2	`ROOTFS`	`ext3`	Kalan alan	Linux dosyaları Linux dosya sistemi ister

GParted'ı kur ve aç:

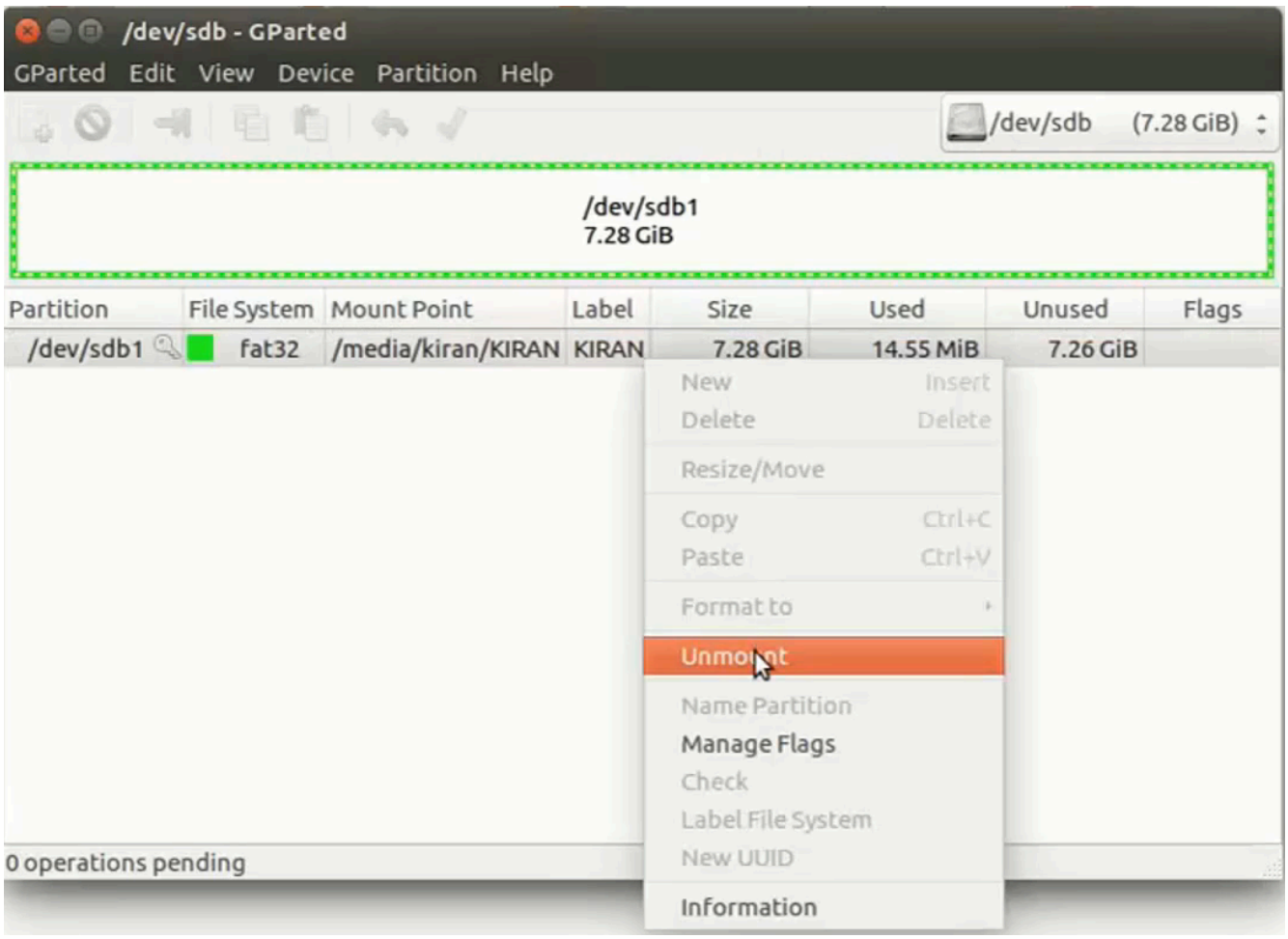
```
sudo apt install gparted
```

> **Dikkat**
> Sağ üstteki listeden **SD kartı seç** (`/dev/sdb` , 7.28 GiB). Yanlışlıkla bilgisayarının diskini (`/dev/sda`) seçersen **bilgisayarındaki veriler silinir**.



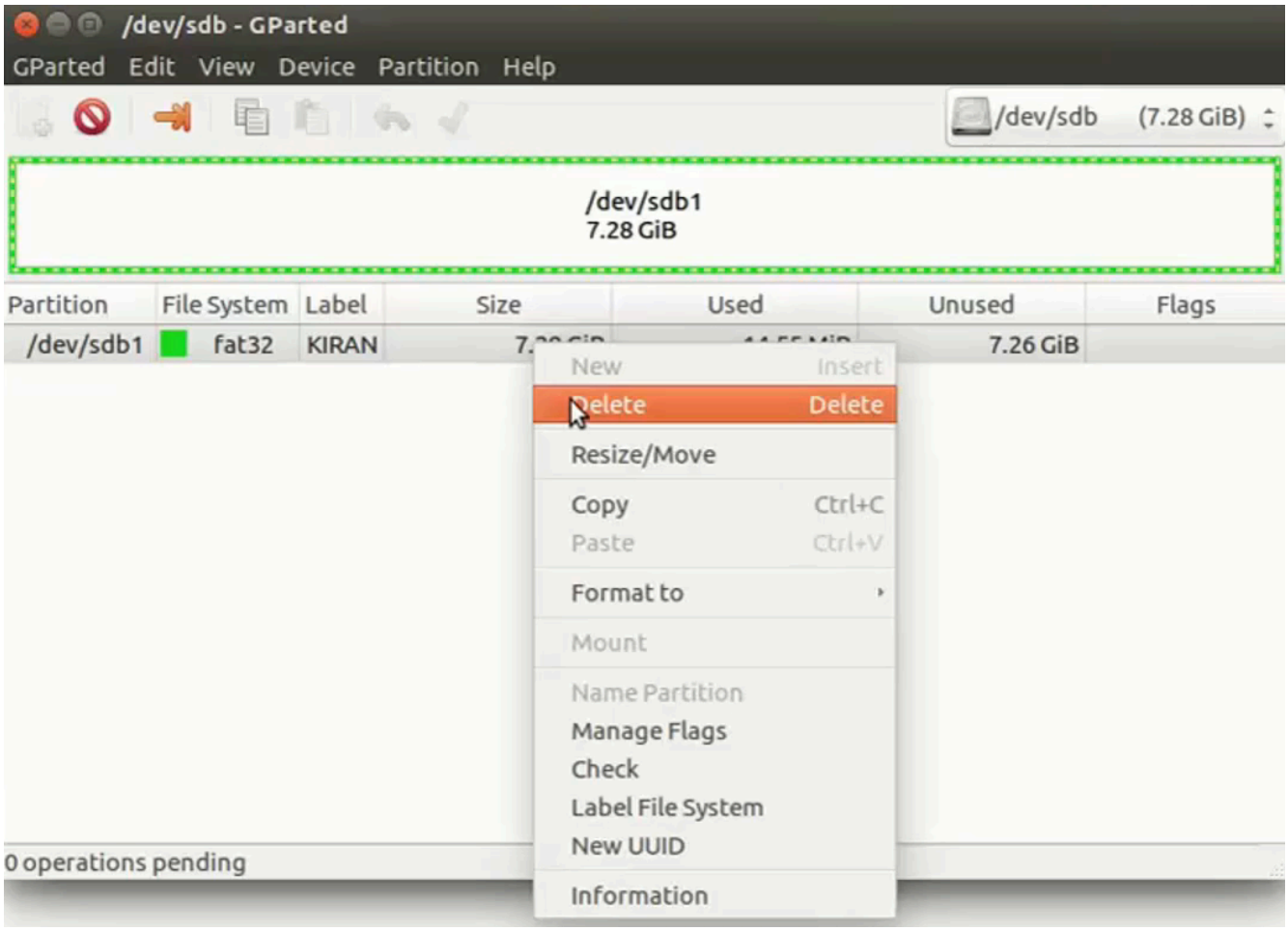
3.1 Unmount et

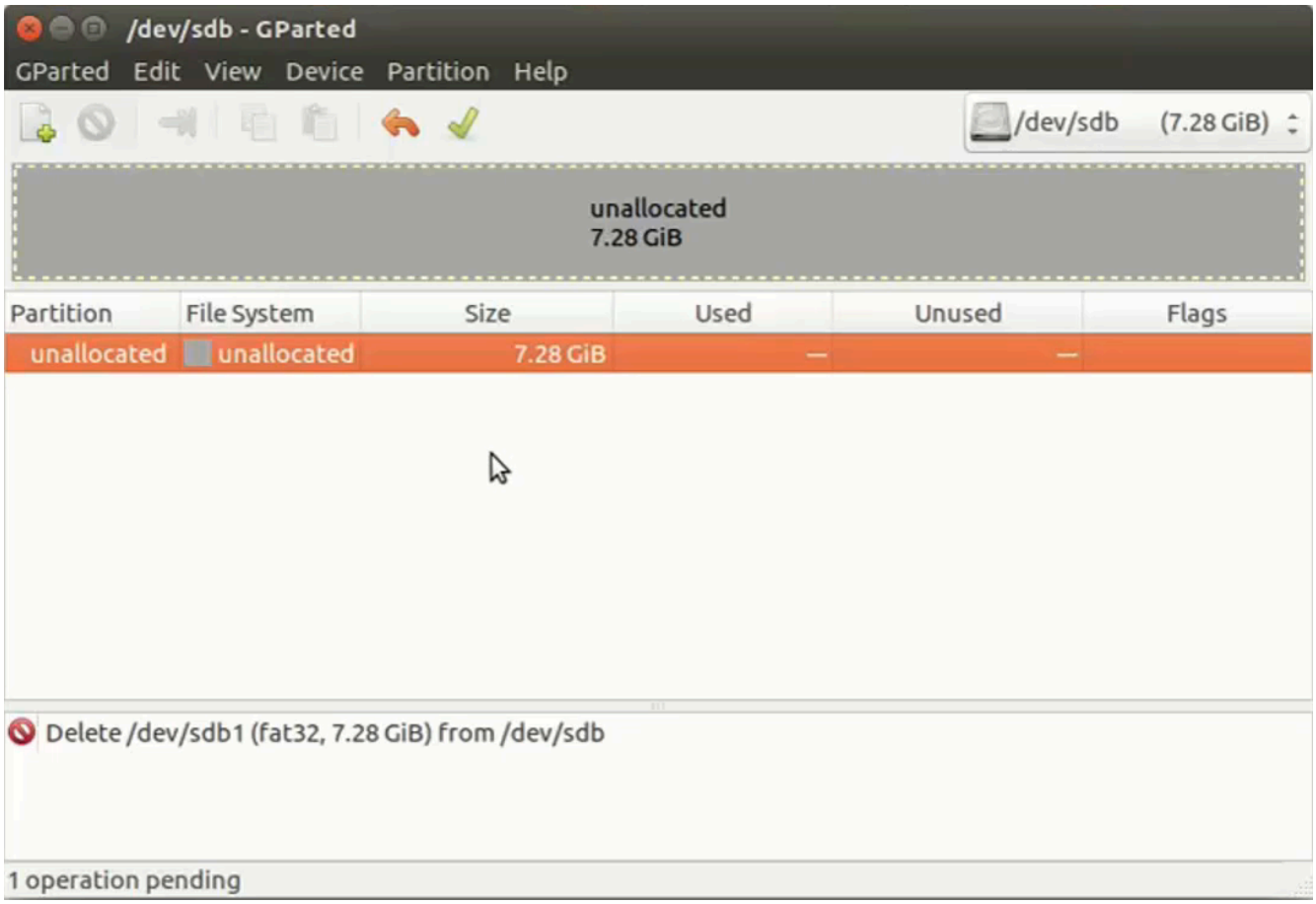
Bölüm şu an sistem tarafından kullanılıyor (yanındaki 🔑 simgesi). Kullanımdayken silinemez. **Sağ tık → Unmount.**



3.2 Eski bölümü sil

Sağ tık → Delete. Kartın tamamı `unallocated` (boş alan) olur.

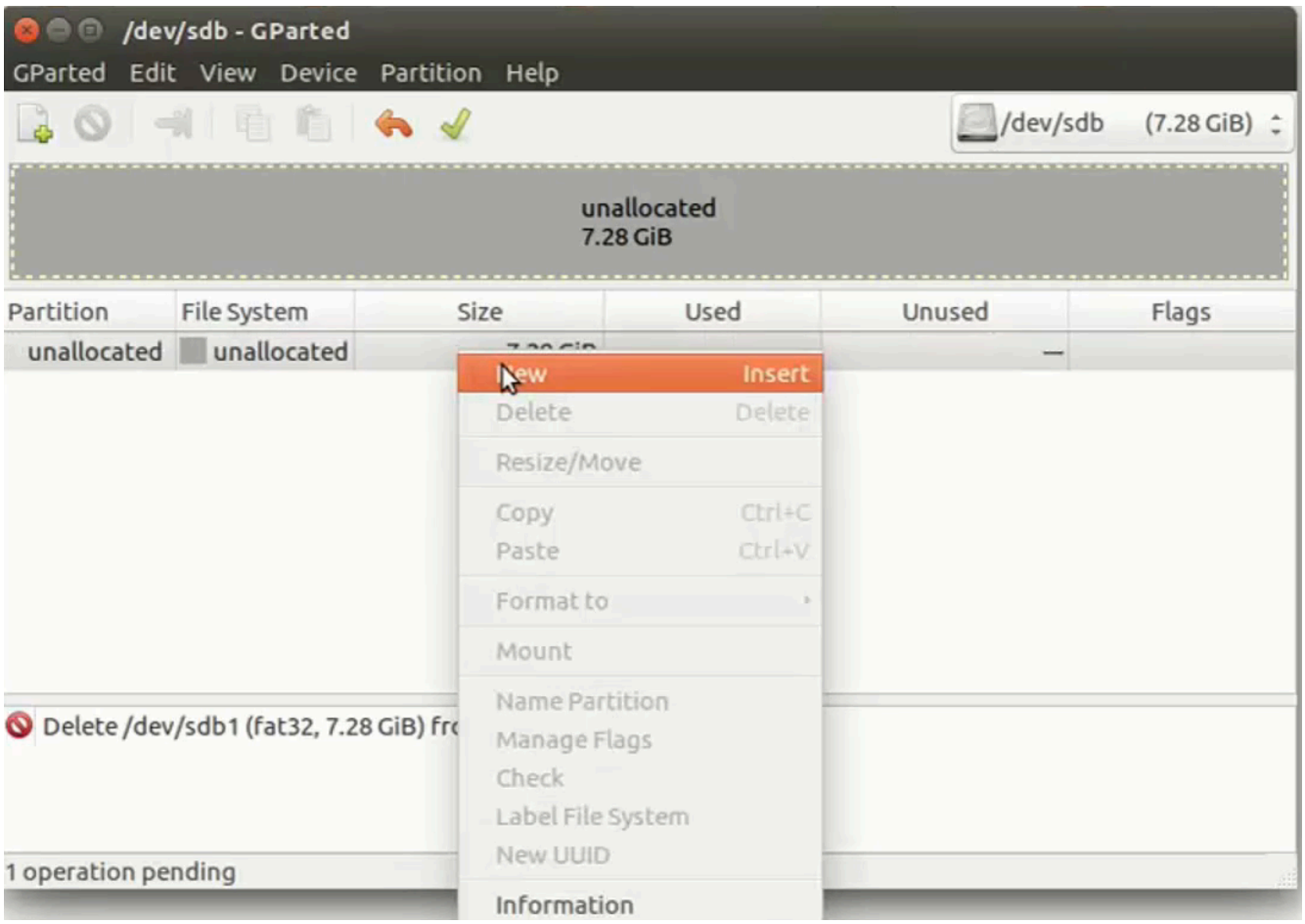

/dev/sdb - GParted
GParted Edit View Device Partition Help
/dev/sdb (7.28 GiB)
/dev/sdb1
7.28 GiB
Partition File System Label Size Used Unused Flags
/dev/sdb1 fat32 KIRAN 7.26 GiB
New Insert
Delete Delete
Resize/Move
Copy Ctrl+C
Paste Ctrl+V
Format to
Mount
Name Partition
Manage Flags
Check
Label File System
New UUID
Information
0 operations pending


/dev/sdb - GParted
GParted Edit View Device Partition Help
/dev/sdb (7.28 GiB)
unallocated
7.28 GiB
Partition File System Size Used Unused Flags
unallocated unallocated 7.28 GiB — —
Delete /dev/sdb1 (fat32, 7.28 GiB) from /dev/sdb
1 operation pending

> **Not**
>
> GParted işlemleri hemen uygulamaz, sıraya koyar ("1 operation pending"). Hepsi en sonda 3.5'te uygulanır. O ana kadar her şey geri alınabilir.

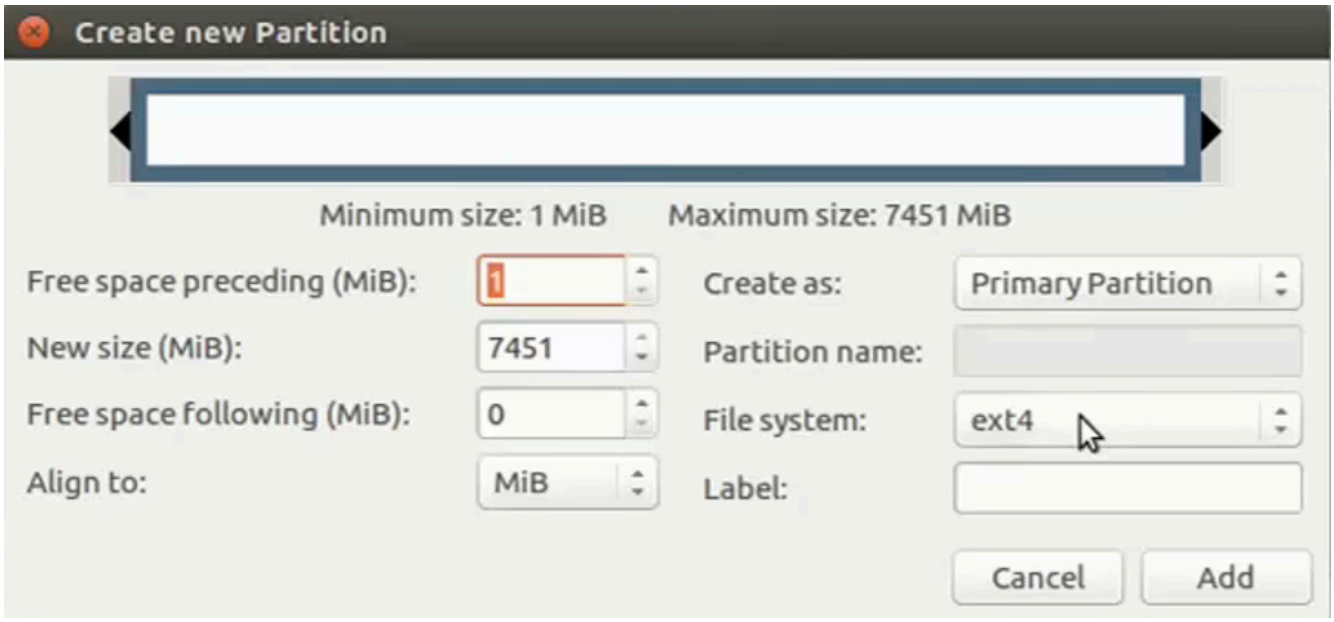
3.3 BOOT bölümünü oluştur

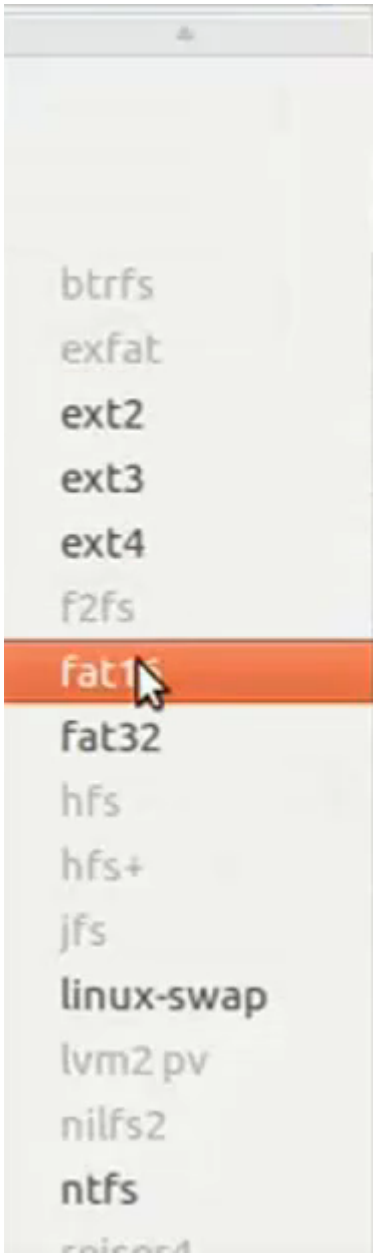
`unallocated` alana **sağ tık → New.**

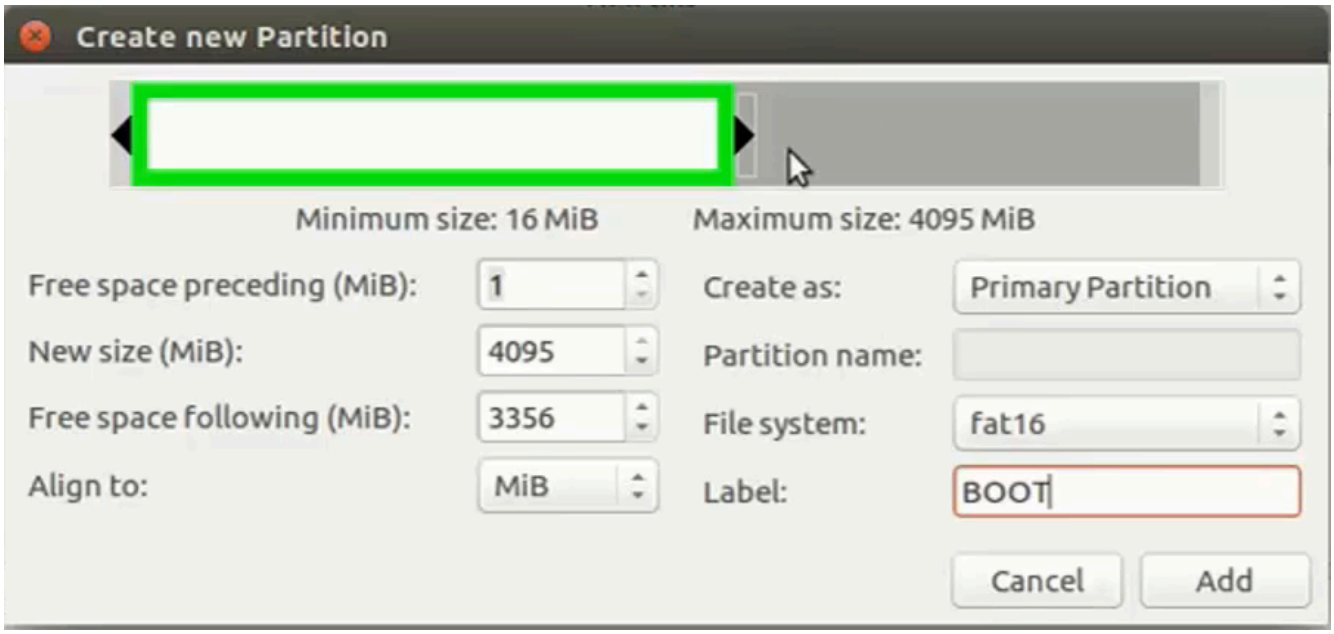


Açılan pencerede 3 ayar yap, sonra **Add**'e bas:

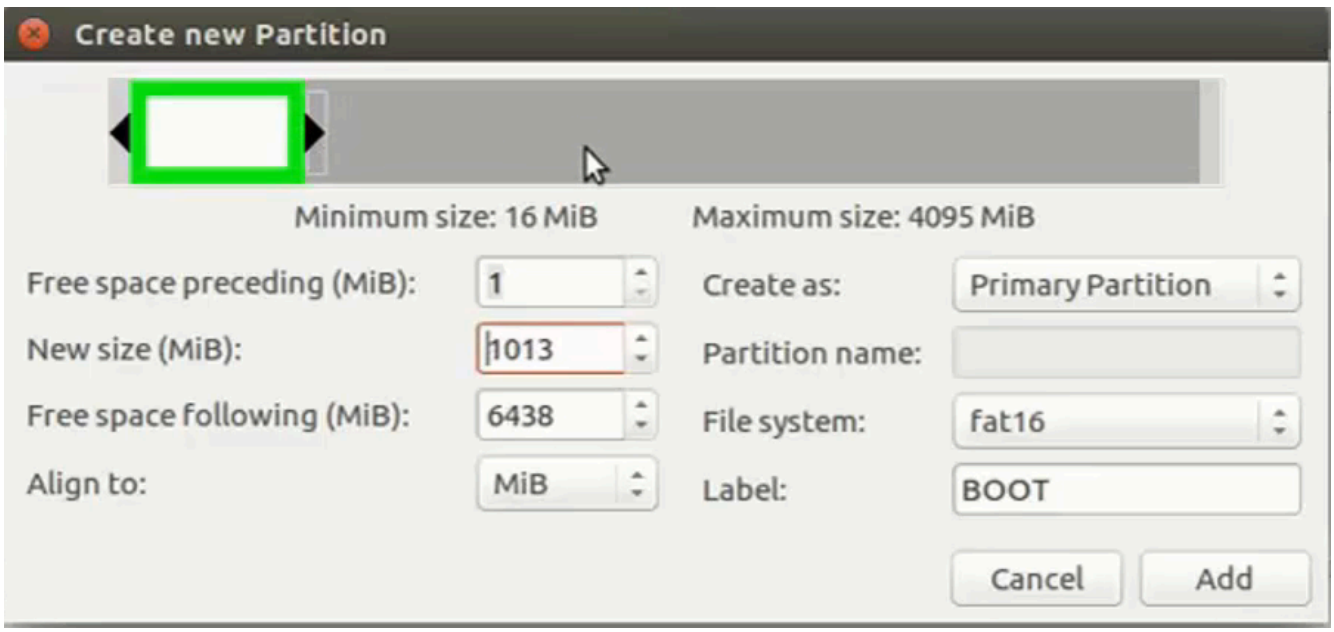
Alan	Değer
File system	`fat16`
Label	`BOOT`
New size (MiB)	`1013` (yaklaşık 1 GB)

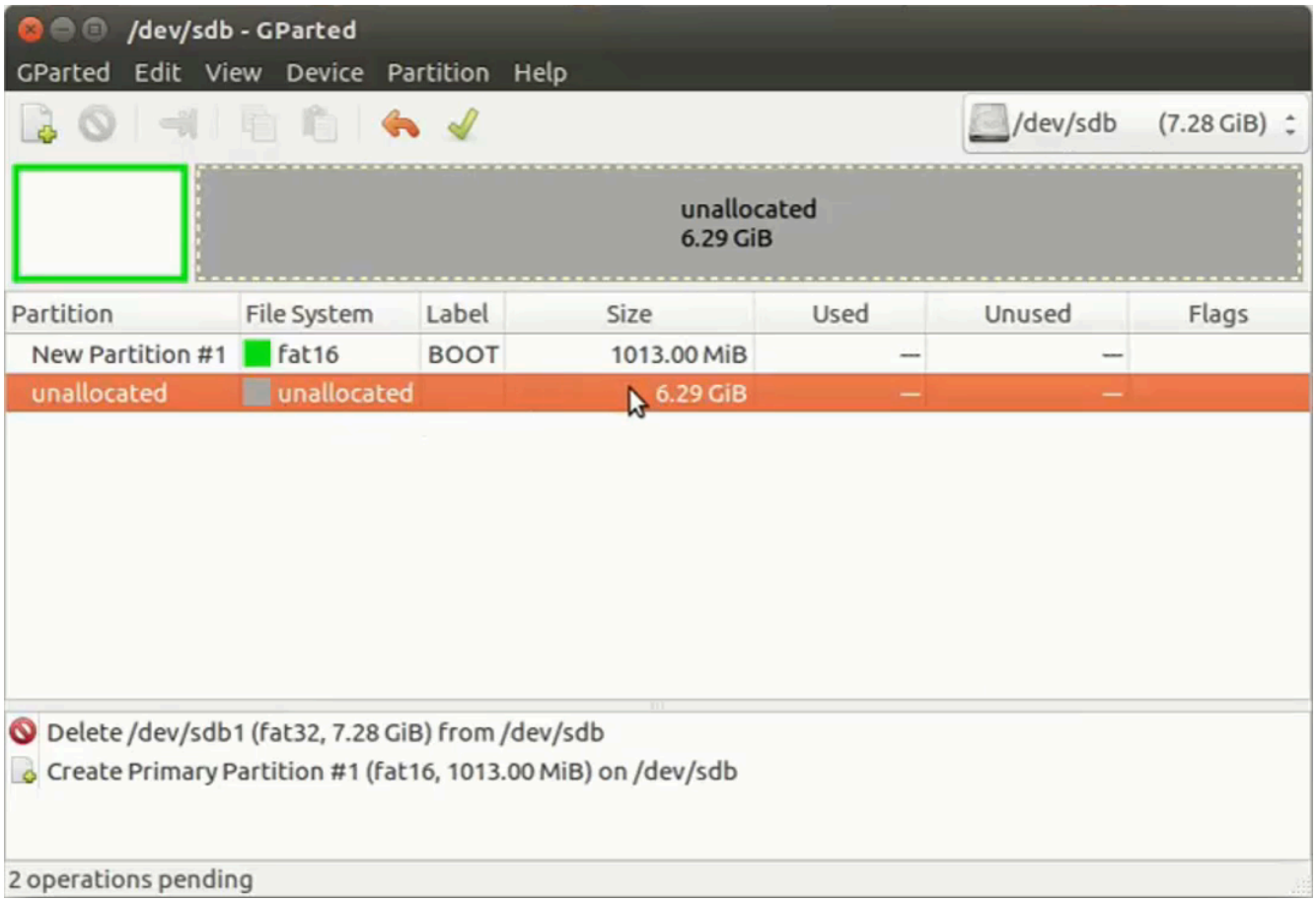

Create new Partition
Minimum size: 1 MiB
Maximum size: 7451 MiB
Free space preceding (MiB):
1
New size (MiB):
7451
Free space following (MiB):
0
Align to:
MiB
Create as:
Primary Partition
Partition name:
File system:
ext4
Label:
Cancel
Add


btrfs
exfat
ext2
ext3
ext4
f2fs
fat16
fat32
hfs
hfs+
jfs
linux-swap
lvm2 pv
nilfs2
ntfs



> **Not**
>
> fat16 seçilince boyut kendiliğinden **4095 MiB** olur, çünkü FAT16 en fazla ~4 GiB olabilir. Bunu 1013'e düşürüyoruz. Boot dosyaları zaten birkaç MB.

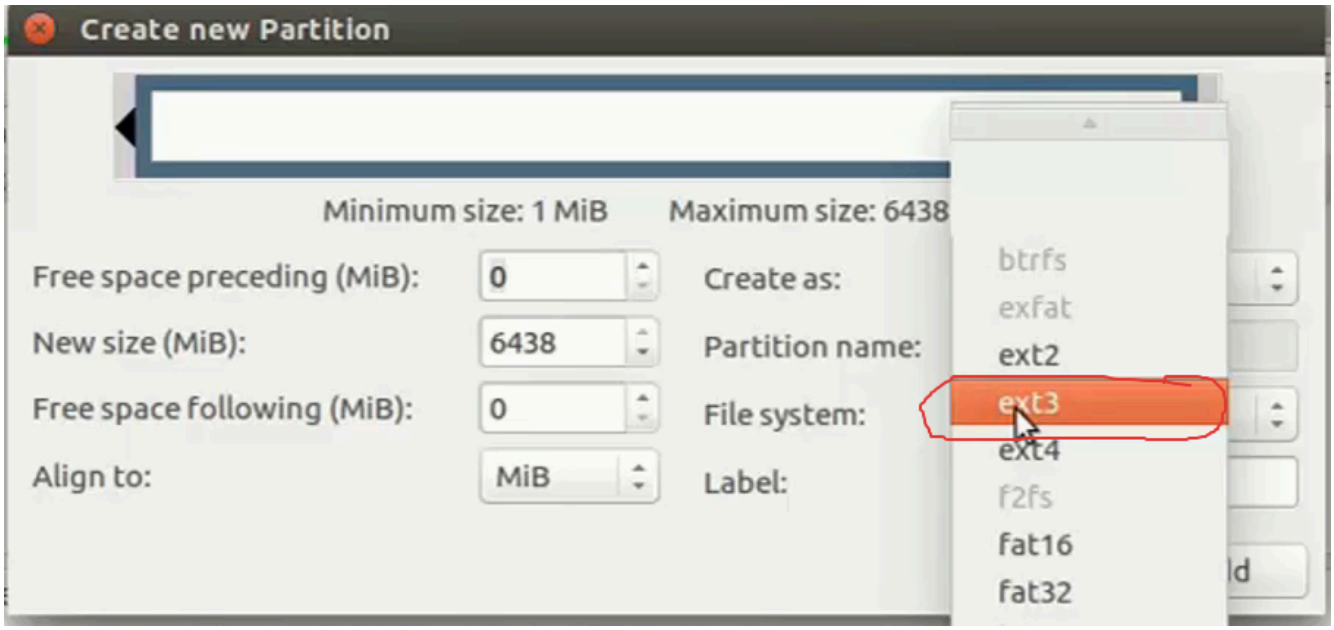


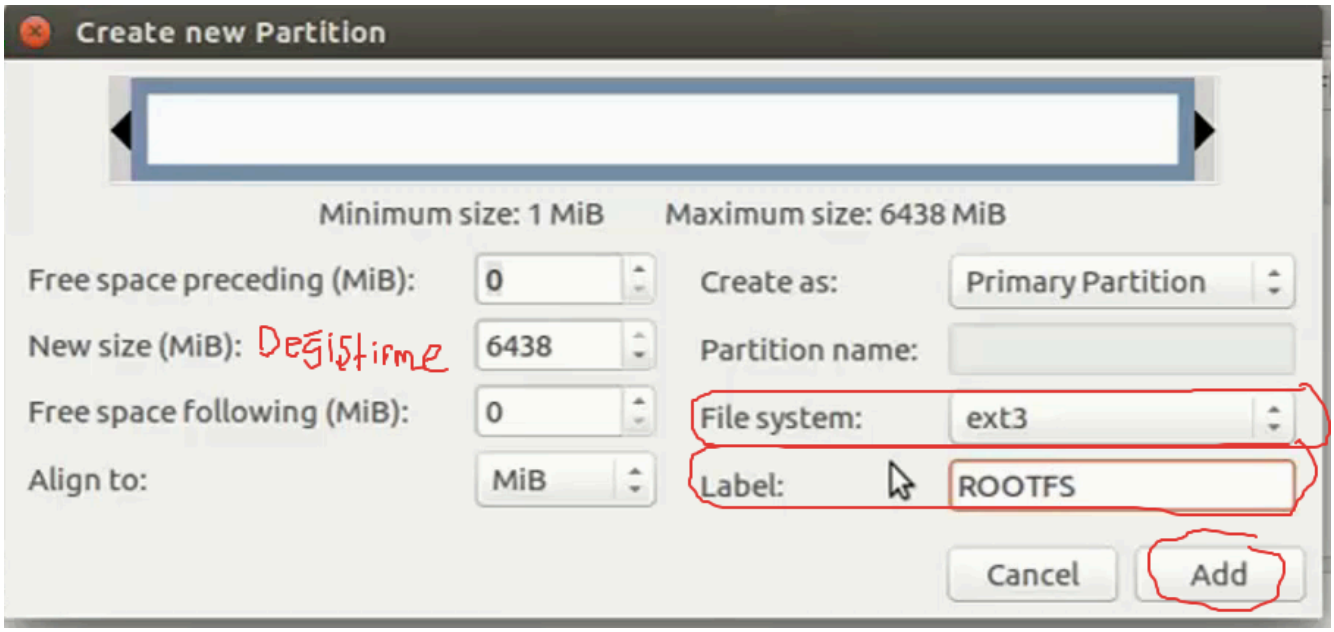


3.4 ROOTFS bölümünü oluştur

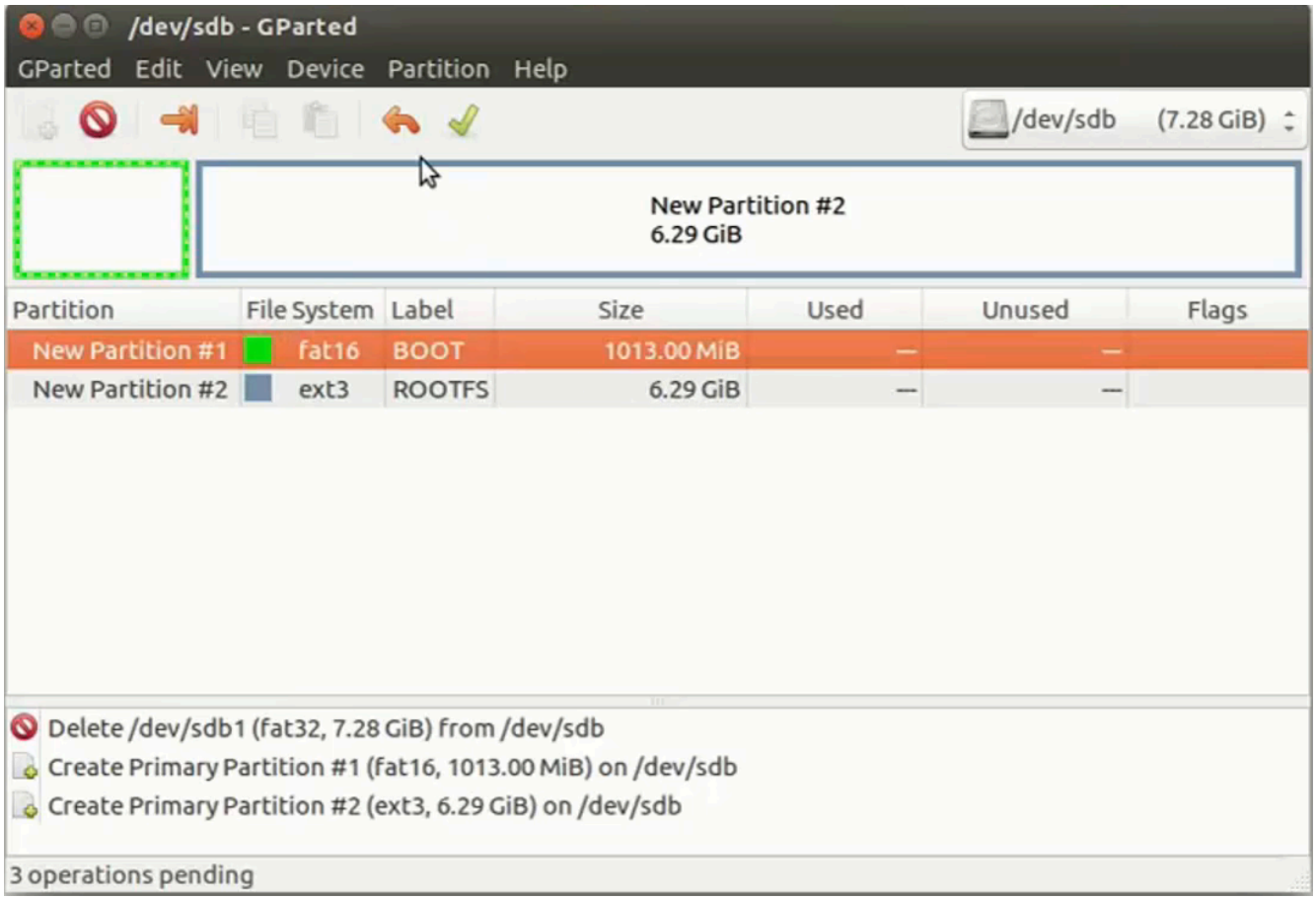
Kalan `unallocated` alana yine **sağ tık** → **New**, sonra **Add**:

Alan	Değer
File system	`ext3`
Label	`ROOTFS`
New size (MiB)	Değiştirme (kalan alanın tamamı)





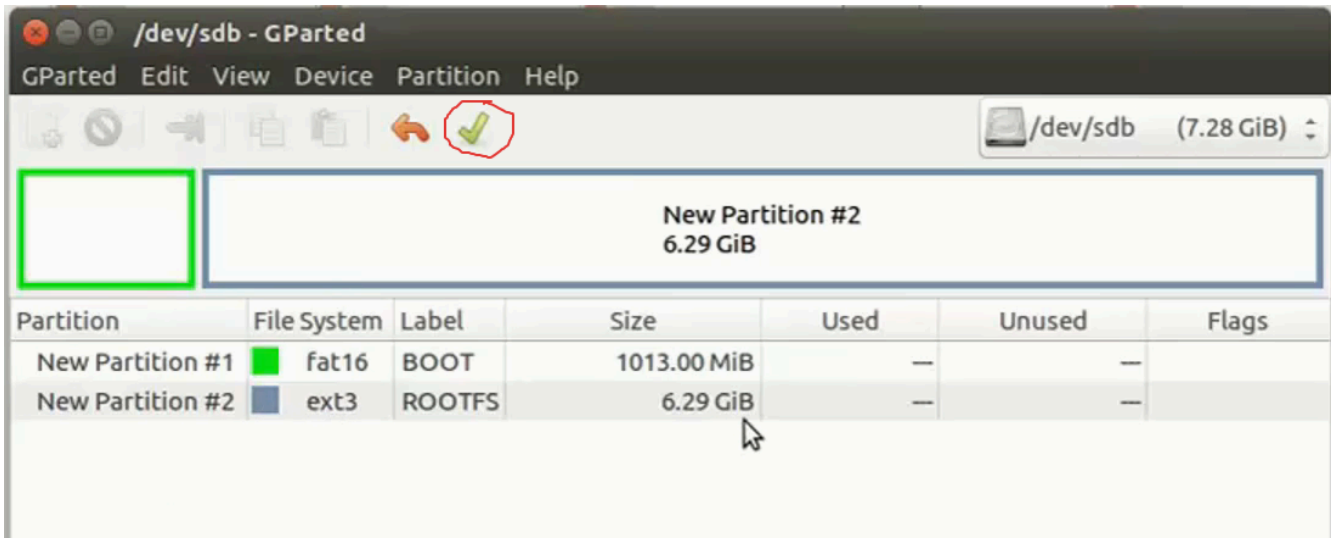
Planlanan durum: 1 GB BOOT + 6.29 GiB ROOTFS.

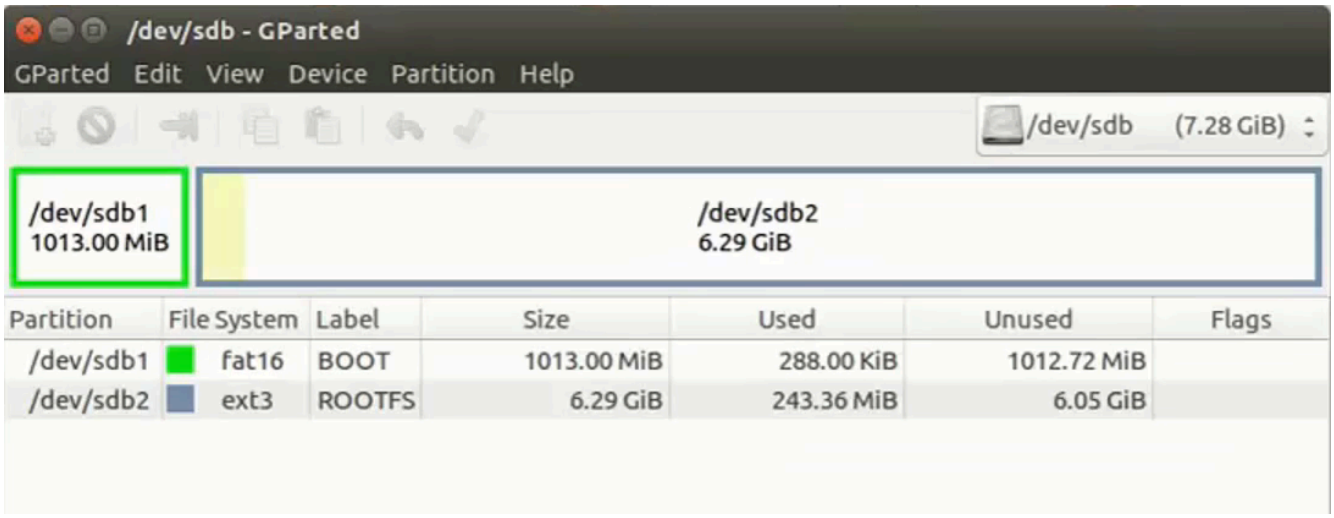


3.5 İşlemleri uygula

Yeşil tik (Apply All Operations) → onay penceresinde **Apply**.

> **Uyarı**
>
> Bu adımdan sonra geri dönüş yok, karttaki eski veriler silinir.

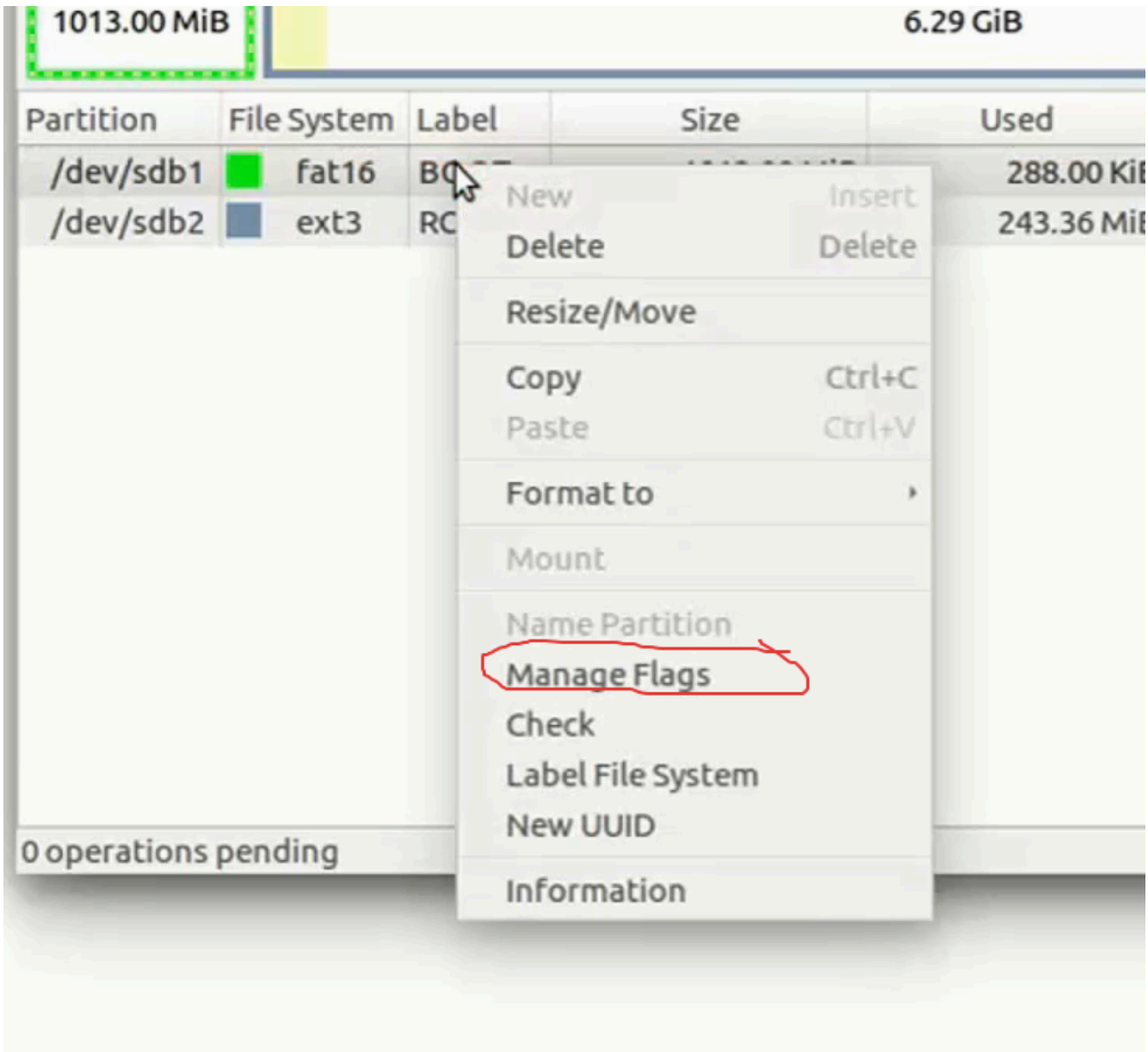


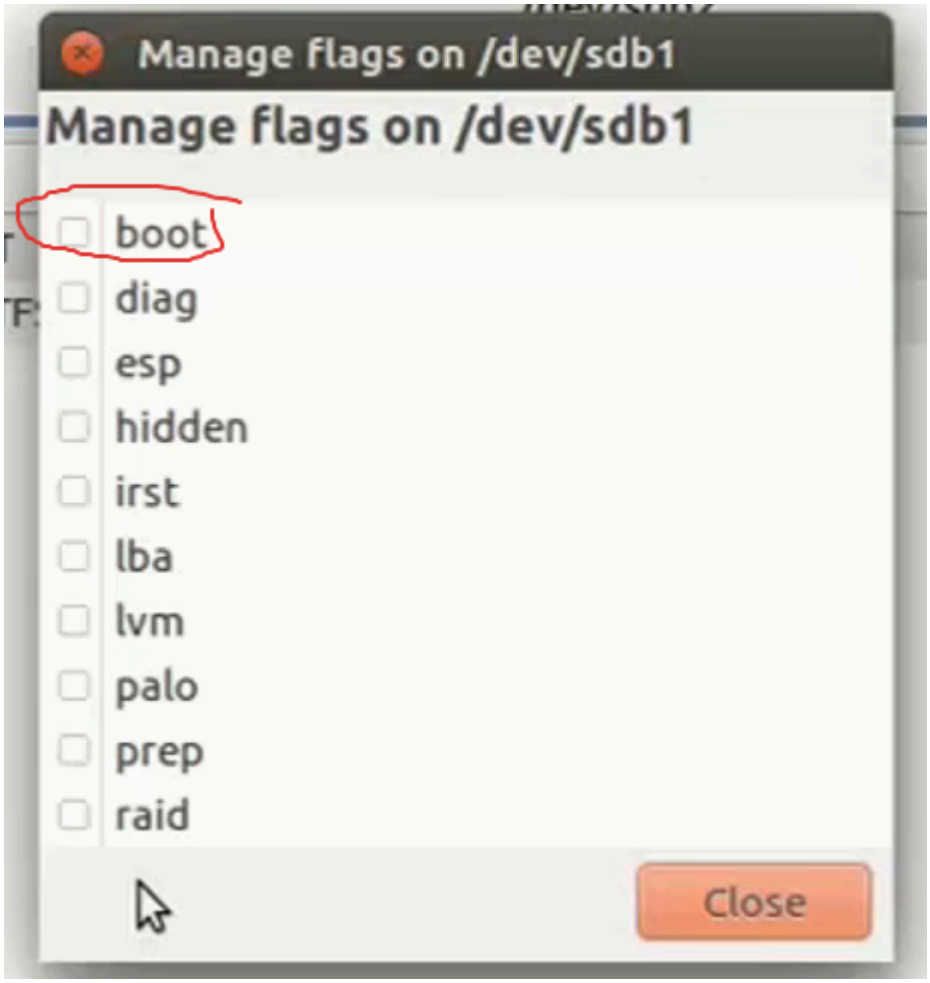


3.6 BOOT bölümüne `boot` bayrağı ekle

Bu bayrak, bölümü "açılış bölümü" olarak işaretler. RBL, MLO'yu bu bölümde arar.

BOOT'a sağ tık → Manage Flags → `boot` kutucuğunu işaretle → Close.





Flags sütununda `boot` yazıyorsa tamam. GParted'ı kapatabilirsin.

Partition	File System	Label	Size	Used	Unused	Flags
/dev/sdb1	fat16	BOOT	1013.00 MiB	288.00 KiB	1012.72 MiB	boot
/dev/sdb2	ext3	ROOTFS	6.29 GiB	243.36 MiB	6.05 GiB	

---

4. Boot Prosedürü (her testte tekrarlanır)

Aşağıdaki aşamaların her birinde dosya kopyaladıktan sonra bu adımları uygula.

4.1 Kartı bilgisayardan güvenli çıkar

```
sync
sudo umount /media/$USER/BOOT
sudo umount /media/$USER/ROOTFS
```

- `sync` → Bekleyen yazma işlemlerini karta yazar. Atlanırsa kopyalanan dosya eksik kalabilir.
- `umount` → Bölümlerin bağlantısını keser. Artık kart çıkarılabilir.

4.2 Seri konsolu hazırla (sadece ilk seferde)

1. USB-TTL'yi kartın **J1** header'ına bağla. Pinler: beaglebone_hardware.md
2. `minicom`'u aç: **115200 baud, 8N1, Hardware Flow Control kapalı**. Ayarlar: minicom.md

4.3 Kartı SD karttan aç (S2 butonu)

BeagleBone Black normalde **kendi eMMC belleğinden** açılır. SD karttan açılması için **kart açılırken S2 (BOOT) butonu basılı olmalı**. RBL bu butona sadece açılış anında bakar.

SD kartı BeagleBone'a tak, sonra iki yöntemden birini uygula:

Yöntem	Ne zaman?	Adımlar
A: Güç vererek	Kartta güç yokken	S2'yi basılı tut → USB kablosunu tak → 2–3 sn sonra S2'yi bırak
B: Power butonuyla	Kart zaten güç alıyorsa	Power'a LED'ler sönene kadar uzun bas (kapanır) → S2'yi basılı tut → Power'a kısa bas (açılır) → S2'yi bırak

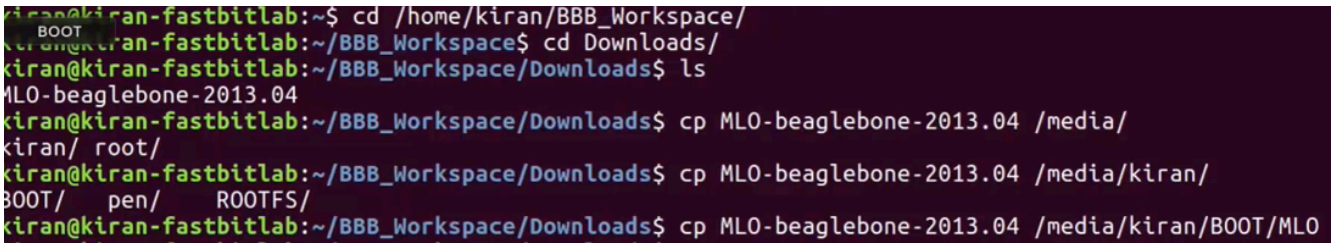
---

5. Aşama 1: MLO

Kopyala

İndirme klasörüne git ve MLO'yu BOOT bölümüne `MLO` **adıyla** kopyala:

```
cd ~/BBB_Workspace/Downloads/
cp MLO-beaglebone-2013.04 /media/$USER/BOOT/MLO
```

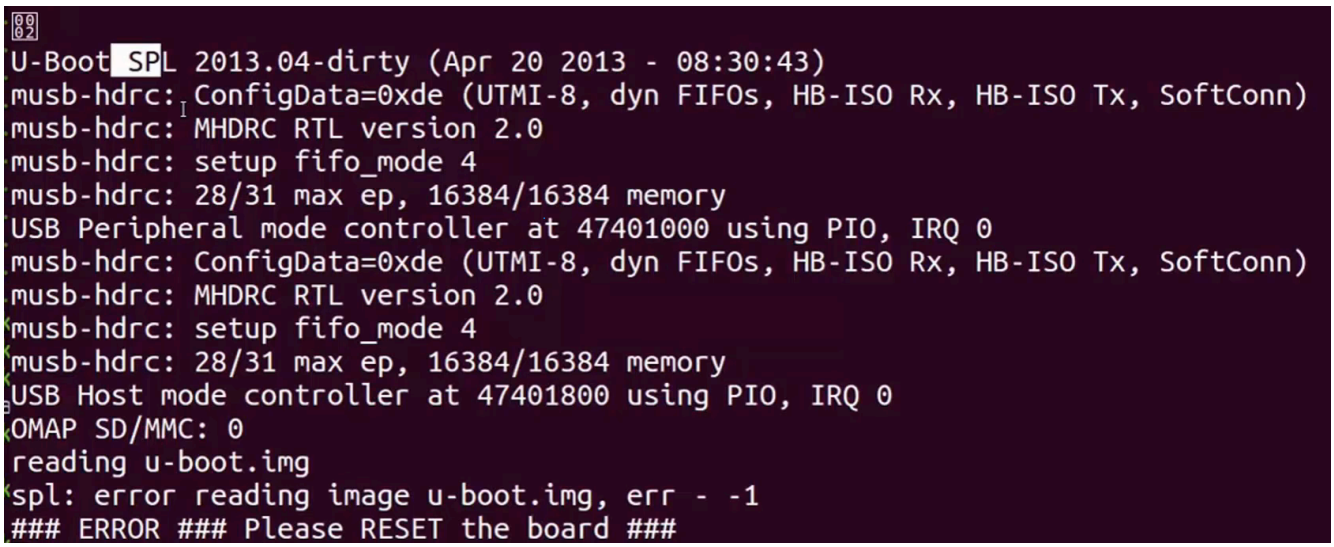


> Önemli
>
> **Neden sonda `MLO` yazıyoruz?** `cp` komutunda hedefin sonuna bir ad yazılırsa dosya o adla kopyalanır:
>
> ```
> cp  MLO-beaglebone-2013.04   /media/$USER/BOOT/MLO
>     └──── kaynak ────┘       └─ hedef klasör + YENİ AD ─┘
> ```
>
> RBL, BOOT bölümünde **tam olarak `MLO` adlı** dosyayı arar. Dosya uzun adıyla kalırsa bulunamaz.

> **İpucu**
> `$USER` senin kullanıcı adındır (kursta `kiran`). Dosya adlarını elle yazmak yerine ilk birkaç harfi yazıp **Tab**'a bas, terminal tamamlar.

Test et

Bölüm 4'teki adımlarla kartı aç. minicom'da şunu görmelisin:



Satır	Anlamı
`U-Boot SPL 2013.04-dirty ...`	**MLO çalıştı.** RBL dosyayı buldu.
`musb-hdrc ...`, `USB ... controller`	MLO USB donanımını başlatıyor, önemli değil.
`reading u-boot.img`	MLO bir sonraki aşamayı (U-Boot) arıyor.
`spl: error reading image u-boot.img`	U-Boot bulunamadı, çünkü henüz kopyalamadık. **Beklenen hata.**

Sonuç: **RBL** ✓ → **MLO** ✓ → **U-Boot** ✗

> **Not**
> "dirty" bir hata değildir. MLO derlenirken kaynak kodda kaydedilmemiş (commit edilmemiş) değişiklik olduğunu belirtir.

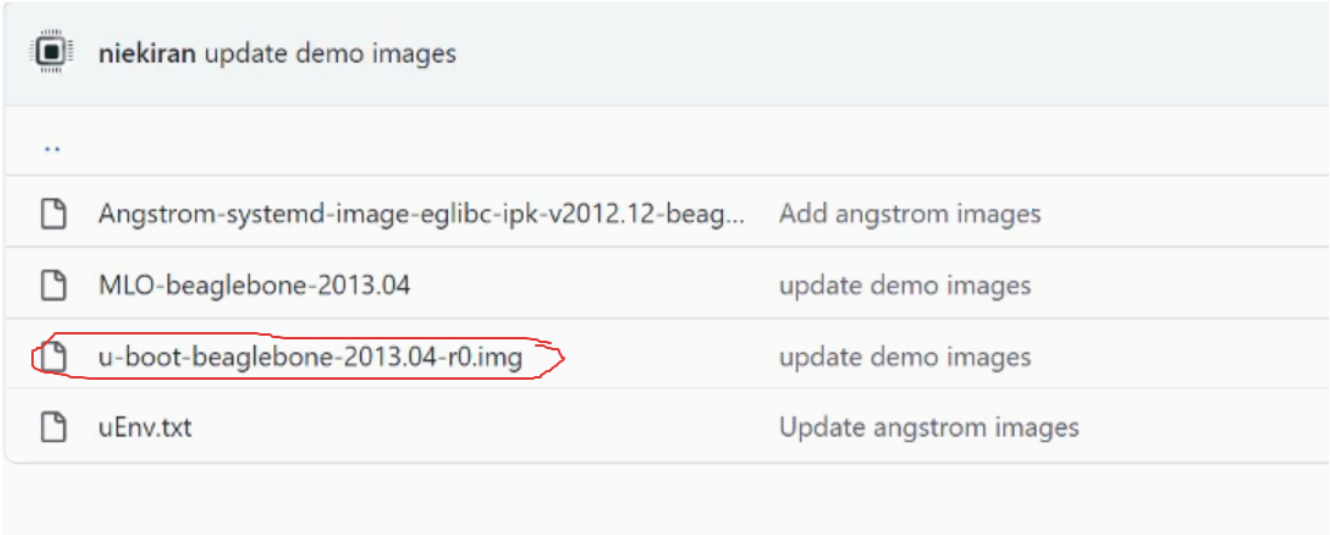
---

6. Aşama 2: U-Boot

Kopyala

Kartı tekrar bilgisayara tak ve U-Boot'u `u-boot.img` **adıyla** kopyala. MLO, U-Boot'u bu adla arar:

```
cp u-boot-beaglebone-2013.04-r0.img /media/$USER/BOOT/u-boot.img
```



Test et

Bölüm 4'teki adımlarla kartı aç.

- `U-Boot SPL` satırlarından sonra **U-Boot'un kendi mesajları** görünüyorsa bu aşama başarılı.
- Kernel ve `uEnv.txt` henüz olmadığı için Linux SD karttan açılmaz.

Sonuç: **RBL** ✓ → **MLO** ✓ → **U-Boot** ✓ → **Kernel** ✗

---

7. Aşama 3: Root Filesystem ve uEnv.txt

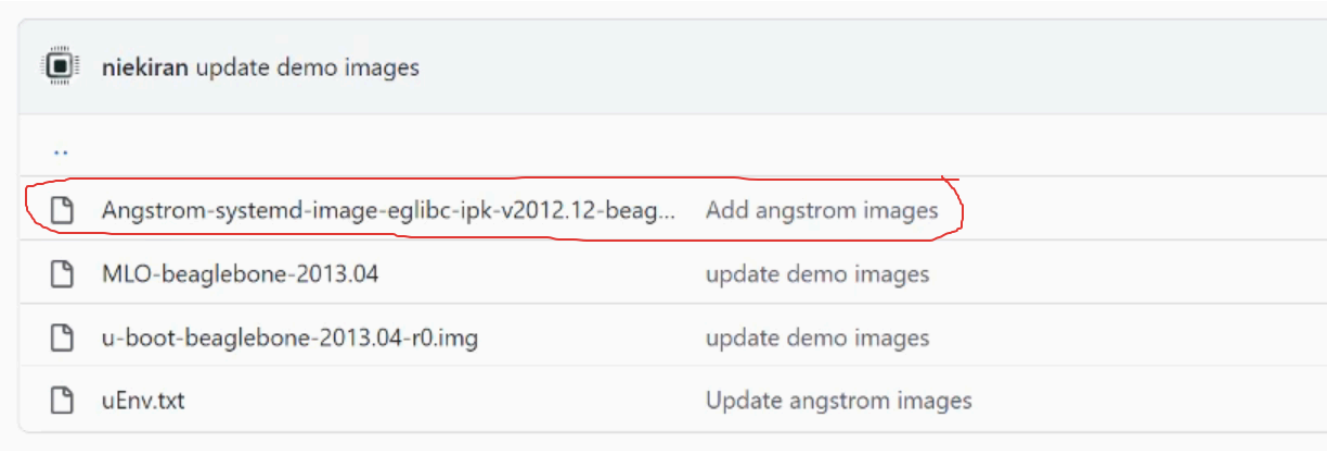
Kopyala

Kartı tekrar bilgisayara tak.

1) Root filesystem'i ROOTFS bölümüne çıkar:

```
sudo tar -xvf Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz -C /media/$USER/ROOTFS
```

Parça	Anlamı
`sudo`	Linux dosyalarının sahibi `root`. Dosya izinlerinin korunması için gerekli.
`-x` `-v` `-f`	Arşivi aç (e**x**tract), açılan dosyaları göster (**v**erbose), arşiv dosyası (**f**ile)
`-C /media/$USER/ROOTFS`	Dosyaları bu klasöre çıkar



2) uEnv.txt'yi BOOT bölümüne kopyala (adı değişmiyor, U-Boot bu adla arar):

```
cp uEnv.txt /media/$USER/BOOT/
```

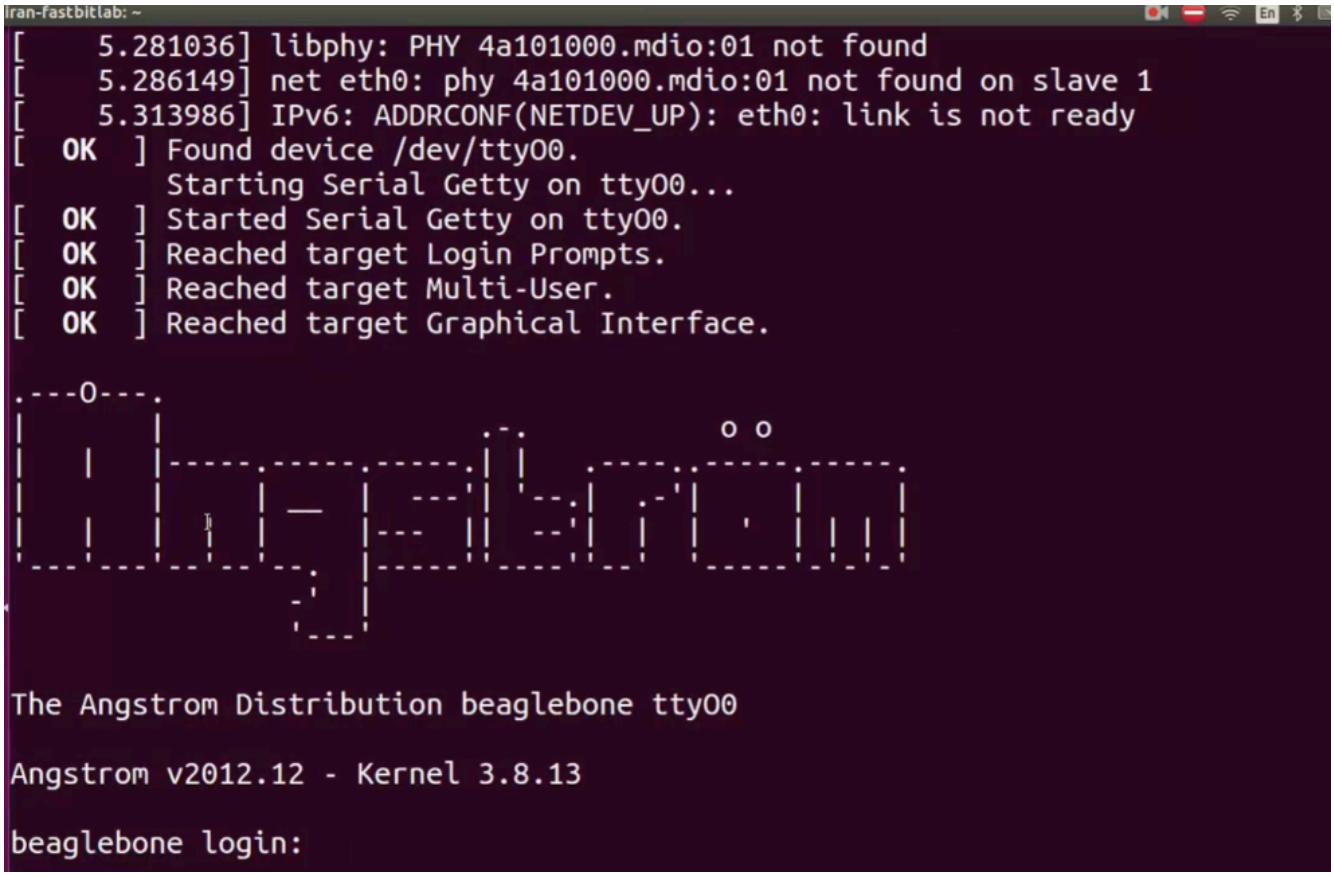
Kontrol et:

```
ls /media/$USER/BOOT/            # MLO  u-boot.img  uEnv.txt
ls /media/$USER/ROOTFS/boot/     # uImage ve am335x-boneblack.dtb burada olmalı
```

Test et

Bölüm 4'teki adımlarla kartı aç. Rootfs büyük olduğu için `sync` biraz uzun sürebilir, bitmesini bekle.

Kernel mesajları akar ve sonunda **Angstrom** logosu ile `beaglebone login:` satırı çıkar:



Sonuç: **RBL ✓ → MLO ✓ → U-Boot ✓ → Kernel ✓ → Rootfs ✓** 🎉

> **İpucu**
> Giriş için kullanıcı adı `root` yaz. Bu demo imajında genelde şifre yoktur.

---

8. Uygulama: uImage Header'ını U-Boot'ta Elle Okuma

Amaç: uImage'ın başındaki 64 byte'lık U-Boot header'ını (Ek Bilgiler → uImage nedir?) kendi gözümüzle görmek.

Neredeyiz? Bölüm 7'de U-Boot, `uEnv.txt` 'deki komutları **otomatik** çalıştırıp Linux'u açtı. Bu sefer U-Boot'u durdurup aynı işin ilk yarısını **elle** yapacağız:

```
Bölüm 7:  RBL → MLO → U-Boot → Kernel → Rootfs → login     (otomatik)
Bölüm 8:  RBL → MLO → U-Boot ■ DUR → komutları elle yaz
```

Adım	Komut	Ne yapar?
1	Açılışta bir tuşa bas	U-Boot'u durdurur, `U-Boot#` komut satırı gelir
2	`load mmc 0:2 0x82000000 /boot/uImage`	uImage'ı SD karttan DDR'e kopyalar
3	`md.b 0x82000000 40`	DDR'deki ilk 64 byte'ı (header) ekrana döker
4	—	Byte'ları header tablosuyla eşleştirip çözeriz

> **Not**
>
> **Neden önce DDR'e yüklüyoruz?** `md` (memory display) komutu sadece **belleği** okuyabilir. SD kart bir bellek adresi değil, depolama cihazıdır. Bu yüzden dosyayı önce RAM'e kopyalıyoruz.

8.1 Seri terminal: minicom veya Tera Term

Kursta bu derste Windows'ta **Tera Term** kullanılıyor. Bu bir zorunluluk değil. Tera Term da minicom gibi bir **seri terminal programı**. Komutları bilgisayar değil, **kartın üzerindeki U-Boot** çalıştırır. Bilgisayardaki program sadece bir penceredir:

```
[Klavye] ⇄ [minicom / Tera Term] ⇄ USB-TTL kablo ⇄ [BeagleBone: U-Boot]
```

	Linux	Windows
Program	`minicom`	Tera Term
Port adı	`/dev/ttyUSB0`	`COM3` gibi (Aygıt Yöneticisi → Bağlantı Noktaları'na bak)
Ayarlar	115200, 8N1, flow control yok	Setup → Serial port: 115200, 8 bit, none, 1 stop, Flow control: none

Hangisini kullanırsan kullan, komutlar ve çıktılar aynıdır.

8.2 U-Boot'u durdur

1. Seri terminali aç ve kartı Bölüm 4.3'teki gibi SD karttan başlat.
2. Ekranda `Hit any key to stop autoboot` yazısı görünür görünmez bir tuşa (ör. **Space**) bas.
3. `U-Boot#` komut satırı geldiyse hazırsın.

> **İpucu**
>
> Geri sayım çok kısa (1–3 saniye). Kaçırırsan Linux açılır. Kartı yeniden başlatıp daha erken bas. Kart açılırken Space'e art arda basmak işe yarar.

8.3 uImage'ı DDR'e yükle

Önce dosyanın orada olduğunu gör:

```
U-Boot# ls mmc 0:2 /boot
```

Sonra DDR'e yükle:

```
U-Boot# load mmc 0:2 0x82000000 /boot/uImage
```

Parça	Anlamı
`load`	Bir dosya sisteminden dosyayı belleğe kopyala
`mmc 0:2`	MMC cihazı 0 (SD kart), 2. bölüm (ROOTFS)
`0x82000000`	DDR'de dosyanın yazılacağı adres (`uEnv.txt` 'deki `loadaddr` ile aynı)
`/boot/uImage`	Bölüm içindeki dosyanın yolu

`mmc X:Y` ne demek?

```
load  mmc 0:2      0x82000000   /boot/uImage
      └─ NEREDEN ─┘ └─ NEREYE ─┘ └─ HANGİ DOSYA ┘
      SD kart,      DDR'deki      partition içindeki
      2. partition  bu adrese     dosya yolu
```

- `X` = **hangi MMC cihazı.** BeagleBone Black'te:
- `mmc 0` → **microSD kart**
- `mmc 1` → **eMMC** (karta lehimli dahili bellek)
- `Y` = **o cihazın kaçıncı partition'ı.** Numaralandırma **1'den** başlar.

> **Önemli**
> `load` her zaman **DDR'e** yükler. `mmc 0:2` dosyanın **nereye** yazılacağını değil, **nereden okunacağını** söyler. Yani "2. partition'a yükle" değil, "**2. partition'dan oku**, DDR'e yükle".

Bizim SD kartımız için:

Yazım	Anlamı	İçinde ne var?
`mmc 0:1`	SD kart, 1. partition	BOOT (FAT16): `MLO`, `u-boot.img`, `uEnv.txt`
`mmc 0:2`	SD kart, 2. partition	ROOTFS (ext3): `/boot/uImage`, `/boot/am335x-boneblack.dtb`, Linux dosyaları
`mmc 1:...`	eMMC'nin partition'ları	Kartla fabrikadan gelen sistem (ör. Angstrom/Debian)

Örnekler:

```
U-Boot# ls mmc 0:1                                  # BOOT bölümünü listele
U-Boot# ls mmc 0:2 /boot                            # ROOTFS'taki /boot klasörünü listele
U-Boot# load mmc 0:1 0x82000000 uEnv.txt            # BOOT'taki uEnv.txt'yi DDR'e yükle
U-Boot# mmc list                                    # U-Boot'un gördüğü MMC cihazlarını
listele
U-Boot# mmc dev 0                                   # SD kartı seç
U-Boot# mmc part                                    # Seçili cihazın partition tablosunu
göster
```

> **Not**
>
> Linux'ta da benzer bir numaralandırma vardır: `/dev/mmcblk0p2` = MMC cihazı 0, partition 2. `uEnv.txt`'deki `root=/dev/mmcblk0p2` ifadesi, U-Boot'taki `mmc 0:2` ile aynı yeri gösterir. Ancak Linux'un numaraları kernel sürümüne göre U-Boot'tan farklı olabilir, bu yüzden ikisinin her zaman aynı olacağını varsayma.

Çıktıda kaç byte okunduğu yazar (`xxxxxxx bytes read`). Bu sayı uImage'ın boyutudur, yani 64 byte header + zImage.

> **Not**
>
> Bu komut `uEnv.txt`'deki `loadfromsd` satırının yaptığı işin aynısı. Fark sadece bu sefer elle yazmamız.

> **İpucu**
>
> `load` komutu tanınmazsa eski adı olan `ext2load mmc 0:2 0x82000000 /boot/uImage` komutunu kullan (ext3 de bu komutla okunur). `ls` yerine de `ext2ls mmc 0:2 /boot` kullanılır.

8.4 Header'ı ekrana dök

```
U-Boot# md.b 0x82000000 40
```

Parça	Anlamı
`md`	memory display: belleği ekrana dök
`.b`	byte byte göster
`0x82000000`	Başlangıç adresi (uImage'ı yüklediğimiz yer)
`40`	Kaç byte gösterilecek. **U-Boot sayıları hex kabul eder:** `0x40` = **64 byte** = header'ın tamamı

Çıktı 4 satırdır. Her satırda adres, 16 byte ve sağda bu byte'ların ASCII karşılığı bulunur.

> **Uyarı**
>
> `md` komutu nokta olmadan (`md 0x82000000 10`) yazılırsa veriyi **4 byte'lık word'ler** halinde gösterir. ARM işlemci **little-endian**, header ise **big-endian** olduğu için byte'lar ters görünür. Örneğin magic number `27051956` yerine `56190527` olarak görünür. Header'ı okurken `md.b` kullanmak daha kolaydır.

8.5 Byte'ları çöz

Çıktının her satırı header'ın şu alanlarına denk gelir:

```
82000000: [27 05 19 56] [.. .. .. ..] [.. .. .. ..] [.. .. .. ..]
            ih_magic      ih_hcrc       ih_time       ih_size
82000010: [.. .. .. ..] [.. .. .. ..] [.. .. .. ..] [..][..][..][..]
            ih_load       ih_ep         ih_dcrc      os arch type comp
82000020: [ ih_name: 32 byte imaj adı, sağdaki ASCII sütununda okunur ...
82000030:   ...                                                      ]
```

Alan	Nasıl kontrol edilir?
`ih_magic`	`27 05 19 56` olmalı. Değilse bu adreste geçerli bir uImage yok.
`ih_time`	Unix zaman damgası. Linux'ta tarihe çevir: `date -d @$((0xXXXXXXXX))`
`ih_size`	zImage boyutu. Ondalığa çevir: `echo $((0xXXXXXXXX))`. Bu değer + 64 = `load` komutunun okuduğu byte sayısı olmalı.
`ih_load`, `ih_ep`	Kernel'in DDR'de çalışacağı adres ve başlangıç noktası
`os`	`05` = Linux
`arch`	`02` = ARM
`type`	`02` = Kernel
`comp`	`00` = sıkıştırma yok. zImage zaten kendini açtığı için U-Boot ayrıca açmaz.
`ih_name`	Sağdaki ASCII sütununda imajın adı okunur

> **İpucu**
> Hex'i ondalığa çevirmek için Windows Hesap Makinesi'nin **Programcı** modunu da kullanabilirsin.

8.6 Sonucu U-Boot'a doğrulat

Elle çözdüğün değerleri U-Boot'un kendi çözümüyle karşılaştır:

```
U-Boot# iminfo 0x82000000
```

`iminfo` komutu header'ı okuyup Image Name, Created, Image Type, Data Size, Load Address ve Entry Point bilgilerini yazar. Ayrıca CRC'yi kontrol eder (`Verifying Checksum ... OK`). Değerler senin bulduklarınla aynı olmalı.

8.7 Linux'u başlat

İşin bitince Linux'u normal şekilde açmak için:

```
U-Boot# boot
```

Ya da kartı yeniden başlat (`reset` veya Power butonu).

Komut özeti

Komut	Ne yapar?
`ls mmc 0:2 /boot`	SD kartın 2. bölümündeki `/boot` klasörünü listeler
`load mmc 0:2 0x82000000 /boot/uImage`	uImage'ı DDR'de `0x82000000` adresine yükler
`md.b 0x82000000 40`	O adresten başlayarak 64 byte'ı byte byte gösterir
`iminfo 0x82000000`	O adresteki uImage header'ını çözüp yazar
`boot`	Normal açılışa devam eder (Linux'u başlatır)
`reset`	Kartı yeniden başlatır
`help`	Tüm U-Boot komutlarını listeler (`help md` gibi tek komut için de olur)

---

9. Sorun Giderme

Belirti	Olası sebep
minicom'da hiçbir şey yok	TX/RX çapraz bağlı mı? Baud 115200 mü? Hardware Flow Control kapalı mı?
Sürekli `CCCCCCCC...`	RBL, MLO'yu bulamadı. Dosya adı tam `MLO` mu? BOOT bölümü FAT mı? `boot` bayrağı var mı?
`error reading image u-boot.img`	BOOT'ta `u-boot.img` yok veya adı yanlış.
Eski Linux açılıyor (Debian vb.)	Kart eMMC'den açıldı. S2 açılış anında basılı değildi.
U-Boot `/boot/uImage` bulamıyor	Rootfs ROOTFS bölümüne çıkarılmamış veya yanlış klasöre çıkarılmış.
`Kernel panic ... Unable to mount root fs`	ROOTFS bölümü boş veya `sync` yapılmadan kart çıkarılmış.

Ek Bilgiler

U-Boot ne yapar?

1. **Donanımı başlatır:** UART, MMC/SD, USB, Ethernet, I2C, NAND...

2. **Kernel'i RAM'e yükler.** Kernel'i farklı kaynaklardan alabilir: SD kart, eMMC, USB, Ethernet (TFTP), seri port, NAND.
3. **Kernel'e boot parametrelerini (bootargs) iletir.** Örneğin rootfs'un nerede olduğu ve konsolun hangi port olduğu.

uEnv.txt satır satır

`uEnv.txt`, U-Boot'un davranışını değiştiren ayar dosyasıdır. U-Boot açılınca bu dosyayı okur ve içindeki `uenvcmd` komutunu çalıştırır.

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
loadaddr=0x82000000 ──→ DDR Temel Adresi
fdtaddr=0x88000000  ──→ DTB File Adresi
loadfromsd=load mmc 0:2 ${loadaddr} /boot/uImage;load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
linuxbootargs=setenv bootargs console=${console} root=/dev/mmcblk0p2 rw
uenvcmd=setenv autoload no; run loadfromsd; run linuxbootargs; bootm ${loadaddr} - ${fdtaddr}
```

Satır	Anlamı
`console=ttyO0,115200n8`	Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın.
`ipaddr`, `serverip`	Ağdan boot için IP adresleri. SD karttan boot'ta kullanılmıyor.
`loadaddr=0x82000000`	Kernel'in (uImage) **RAM'de (DDR) yükleneceği adres**.
`fdtaddr=0x88000000`	Device tree (.dtb) dosyasının RAM'de yükleneceği adres.
`loadfromsd=load mmc 0:2 ...`	`mmc 0:2` = SD kart, 2. bölüm (ROOTFS). Oradaki `/boot/uImage` → `loadaddr`'e, `/boot/am335x-boneblack.dtb` → `fdtaddr`'e yüklenir.
`linuxbootargs=...` `root=/dev/mmcblk0p2 rw`	Kernel'e "root filesystem SD kartın 2. bölümünde, okuma-yazma modunda" bilgisini verir.
`uenvcmd=...`	U-Boot'un çalıştırdığı komut: önce `loadfromsd`, sonra `linuxbootargs`, en son `bootm` ile kernel'i başlat. `bootm`'deki `-` "initramfs yok" demek.

> **Not**
> AM335x'te DDR'in başlangıç adresi `0x80000000`'dır. `0x82000000` ise DDR'in içinde, kernel için seçilmiş bir adres.

Aynı dosya farklı yazılarak U-Boot'a kernel'i **ağdan (TFTP)**, rootfs'u da **bilgisayardaki bir klasörden (NFS)** aldırmak mümkün. Bu yöntem kursun ilerleyen bölümlerinde kullanılır:

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
absolutepath=/var/lib/tftp/
rootpath=/srv/nfs/bbb,nolock,wsize=1024,rsize=1024 rootwait rootdelay=5
loadtftp=echo Booting from network ...;tftpboot ${loadaddr} ${absolutepath}uImage; tftpb
netargs=setenv bootargs console=${console} root=/dev/nfs rw nfsroot=${serverip}:${rootpa
uenvcmd=setenv autoload no; run loadtftp; run netargs; bootm ${loadaddr} - ${fdtaddr}
```

uImage nedir?

`uImage`, U-Boot'un `bootm` komutuyla başlatabildiği kernel formatıdır. Yeni bir derleme yapılmaz. `zImage` dosyasının başına **64 byte'lık bir U-Boot header'ı** eklenir. Bu işi `mkimage` aracı yapar.

```
+------------------+--------------------------------------+
| U-Boot Header    |               zImage                 |
|   (64 byte)      | (sıkıştırılmış, kendini açan kernel) |
+------------------+--------------------------------------+
|<----------------------- uImage ------------------------>|
```

Kernel imajları zinciri:

Dosya	Açıklama
`vmlinux`	Derlenmiş kernel. ELF formatında, sıkıştırılmamış.
`Image`	`vmlinux`'tan `objcopy` ile çıkarılmış ham binary
`zImage`	Sıkıştırılmış `Image` ve başında onu RAM'de açan küçük bir program
`uImage`	64 byte U-Boot header + `zImage`

Header'ın içeriği (`struct image_header`, tüm alanlar big-endian):

Offset	Boyut	Alan	Anlamı
0x00	4	`ih_magic`	Sihirli sayı: `0x27051956`
0x04	4	`ih_hcrc`	Header'ın CRC32 değeri
0x08	4	`ih_time`	Oluşturulma zamanı
0x0C	4	`ih_size`	zImage'in boyutu
0x10	4	`ih_load`	RAM'de kopyalanacağı adres
0x14	4	`ih_ep`	Çalıştırmanın başlayacağı adres (entry point)
0x18	4	`ih_dcrc`	zImage'in CRC32 değeri
0x1C	1	`ih_os`	İşletim sistemi (Linux)
0x1D	1	`ih_arch`	Mimari (ARM)
0x1E	1	`ih_type`	İmaj tipi (kernel, ramdisk, script...)
0x1F	1	`ih_comp`	Sıkıştırma tipi
0x20	32	`ih_name`	İmaj adı

U-Boot bu header sayesinde dosyanın geçerli bir imaj olup olmadığını (magic number), bozulup bozulmadığını (CRC) ve RAM'de nereye yüklenip nereden çalıştırılacağını anlar.